LLMs

Definition

- LLms are advanced AI models, created to comprehend and produce human language, code and beyond
- Large = billions of parameters, trained on hundreds of billions of words
- Built on the Transformer architecture (2017)
- Versatile in tasks, they excel in accuracy, fluency and style
- Examples: GPT-4, Claude, Gemini, LLaMA, BERT
- A single model learns language patterns so well that it can be reused for many tasks

Conditions

LLMs are DNNs for language and must satisfy all the following:

1. Deep Transformer-based architecture (not RNN, LSTM, …)
2. Very large scale (hundreds of millions to billions of parameters)
3. Pre-trained on massive corpora (books, web, code, …)
4. General-purpose (not task-specific)

Tasks

- Generation: Write essays, emails, stories, reports and code
- Understanding: Classify text, analyse sentiment and recognise named entities
- Transformation: Translate, summarise, rewrite and change writing style
- Reasoning: Support problem solving, maths, planning and decision-making
- Conversation: Power chatbots, tutoring systems and customer support
- Tool use and agents: Call APIs, browse the web, write and run code and complete multi-step tasks

Popular LLMs

- Anthropic's Claude: Reasoning, coding, long context, image understanding
- OpenAI's GPT: General-purpose use, multimodal input, image generation
- Google's Gemini: Long context, multimodal input, image generation, Google integration

Open-weight (free to download & run)

- Meta's LLaMA: Most widely used open base model
- DeepSeek: Very strong reasoning, open weights
- Alibaba's Qwen: Multilingual, strong on Chinese + code
  (Qwen is shortened from Qianwen, meaning "a thousand questions")

Transformer Architecture

Before Transformers: RNNs and LSTMs

- Older models such as RNNs and LSTMs process text 1 token at a time, from left to right
  - Each step depends on the previous step, so training is difficult to parallelise
  - Information from earlier words or earlier sentences may become weaker by the time the model reaches the later next
- Need: An architecture that can look at all tokens in the given context together and learn relationships between distant words more effectively

Transformer Architecture:

- A neural network architecture used in modern LLMs
- Uses self-attention to connect related words / tokens
- Can process text in parallel
- Better at handling long-range relationships in text
- There are 6 layers encoder followed by 6 layers decoder

Encoder

1. Input embeddings + Positional Encoding
   → Turns words into vectors and adds info about word order
2. Multi-head Self-Attention
   → Each word looks at other words in the input to understand the context
3. Feedforward Network
   → A small neural network that transforms each word's representation
4. Add & Layer Norm
   → The input vector is added to the output of the small neural network, and the result is then normalized. Helps with stability and learning

Input and Position

The Transformer uses mathematical positional vectors, not simple numbers like 1, 2, 3 sothe model can understand word order and distance between words

Self-Attention: Q, K, V

- Self-Attention uses 3 learned matrices: Query (Q), Key (K) and Value (V). They are a bit like CNN filters because they learn useful patterns
- The input vectors are processed using the learned Q, K and V matrices to produce contextual output vectors
- Multi-head attention means the Transformer uses multiple sets of Q, K and V matrices in parallel, so different heads can learn different word relationships
- The encoder output is a contextual representation that includes word meaning, word order and relationships with other words

Example of Attention

Sentence: “The cat sat on the mat because it was tired”

Each word has its own Q, K and V vectors:

- Query (Q) = what this word is looking for
- Key (K) = what this word can match with
- Value (V) = the information this word can pass on

What does “it” refer to?

- The word “it” sends out a query: “I’m a pronoun, which noun am I?”
- “cat” advertises a Key that strongly matches
  → High attention weight
- “mat” has a weaker matching Key
  → Low attention weight
- “it” pulls mostly cat’s Value into its own vector

Result: The representation of “it” now contains the meaning of “cat”
This happens in parallel for every word in the sentence

Decoder

- During the training, the whole sentence is input to the decoder at once, but a mask hides future words.
- This lets the model predict all next words in parallel, while only using previous words for each prediction

BERT (Encoder-only)

- Sees the whole sentence at once
- Good for understanding tasks
- Classification, NER, search, Q&A
- Does NOT generate text on its own

GPT (Decoder-only)

- Reads left → Right only (casual mask)
- Good for generation tasks
- Chat, writing, code, completion
- Powers ChatGPT, Claude, Gemini, LLaMA

Training LLMs

Pre-training

- Every LLM on the market has been pre-trained on a large corpus of text data and on specific language modeling-related tasks
- During pre-training, the LLM tries to learn and understand general language and relationships between words
- Every LLM is trained on different corpora and on different tasks
- BERT was originally pre-trained on 2 public available text corpora:
    - English Wikipedia: A comprehensive collection from Wikipedia's English articles, spanning diverse topics and styles, encompassing about 2.5 billion words
    - BookCorpus: An extensive dataset of both fiction and nonfiction books, covering various genres, totaling around 800 million words
- Modern LLMs are trained on dataset vastly larger, thousands of times the size of traditional ones
- BERT was pre-trained on 2 specific language modeling tasks:
    - Masked Language Modeling (MLM) task (auto-encoding task): Helps BERT recognize token interactions within a single sentence
    - Next Sentence Prediction (NSP) task: Helps BERT understand how tokens interact with each other between sentences

Transfer Learning

- Transfer learning is a ML technique that harnesses knowledge from 1 task to enhance performance in a related task
- For LLMs, it involves fine-tuning a pre-trained model for a specific task, like text classification or generation, utilizing its already acquired language knowledge
- This approach significantly reduces training time and resources compared to training models from scratch

Fine-Tuning

- Fine-Tuning involves training the LLM on a smaller, task-specific dataset to adjust its parameters for the specific task at hand
- This allows the LLM to leverage its pre-trained knowledge of the language to improve its accuracy for the specific task
- Fine-Tuning has been shown drastically improve performance on domain-specific and task-specific tasks and lets LLMs adapt quickly to a wide variety of NLP applications

- Steps:
    - Define the model we want to fine-tune as well as any fine-tuning parameters (like learning rate)
    - Aggregate some training data (the format and other characteristics depend on the model we are updating)
    - Compute losses (a measure of error) and gradients (information about how to change the model to minimize error)
    - Update the model through backpropagation (a mechanism to update model parameters to minimize errors)

Limitations and Challenges

1. Hallucination: Confidently make up facts
2. Knowledge cutoff: Only knows what was in training data
3. Context Window: Limited memory per conversation (though growing fast)
4. Cost & Compute: Training and serving are expensive and energy-hungry
5. Bias & Safety: Reflects biases in training data, can produce harmful output
6. Reasoning gaps: Math, multi-step logic, planning are still imperfect
7. Reproducibility & Evaluation: Hard to benchmark “understanding”
8. Data & Copyright: What was the model trained on? And is that OK?